

AMFlow Project Log: Bubble, One-Loop, and Two-Loop Kernels

May 29, 2026

1 Scope

This note documents the current AMFlow sandbox project. The goal is to build up confidence step by step before attempting the two-loop delta-constrained counterterm $I_C(x, y)$. The present project contains:

1. a working ordinary one-loop bubble sanity check;
2. a direct cut-linear one-loop collinear-kernel attempt, which currently fails inside AMFlow;
3. a working uncut linear-propagator GaugeLink setup;
4. a discontinuity reconstruction of the one-loop delta constraint from two uncut linear-propagator runs;
5. a one-loop auxiliary-denominator test where the extra denominator is included in the family but requested with exponent zero;
6. a first two-loop GaugeLink setup that reconstructs the two delta constraints from four uncut sign-prescription pieces.

The two-loop setup is still at the first low-order AMFlow-test stage. The purpose is to check whether AMFlow can reduce and evaluate the four uncut two-loop GaugeLink pieces before increasing the requested Laurent order.

2 Directories and External Setup

The project directory is

```
/Users/FranzkeMM/Documents/Dokumente privat/Studium/ETH Zürich/  
Master Thesis/master
```

The AMFlow entry point is the subdirectory `amflow-project`; the local Python comparison code is in the sibling subdirectory `local-subtraction-method`. The AMFlow package is kept outside iCloud:

```
/Users/FranzkeMM/physics/tools/AMFlow
```

The helper file

```
/Users/FranzkeMM/physics/tools/amflow-paths.wl
```

adds the relevant paths for FiniteFlow, LiteIBP, LiteRed, and the FiniteFlow dynamic library. A separate AMFlow-specific fix was also required: the reducer file

AMFlow/ibp_interface/FiniteFlow+LiteRed/install.m

must set

\$FFPath

\$FFLibraryPath

\$LiteIBPPath

\$LiteRedPath

directly, because AMFlow's dependency check does not rely only on \$Path. Finally, fflowmlink.dylib needed an rpath pointing to the FiniteFlow build directory so that it can find libfflow.dylib.

3 Project Structure

The active project files are:

- config/LoadAMFlow.wl: sets absolute project paths, loads amflow-paths.wl, then loads AMFlow.
- config/AMFlowOptions.wl: currently only sets the thread count.
- families/BubbleFamily.wl: ordinary one-loop massive bubble.
- targets/RunBubble.wl: runs the bubble sanity check.
- families/OneLoopKernelDirectCutFamily.wl: direct cut-linear attempt.
- targets/RunOneLoopKernelDirectCut.wl: runs the direct cut-linear attempt.
- families/OneLoopKernelUncutPlusFamily.wl: uncut $+L$ linear denominator family.
- targets/RunOneLoopKernelUncutPlus.wl: runs the $+L$ GaugeLink integral.
- families/OneLoopKernelUncutMinusFamily.wl: uncut $-L$ linear denominator family.
- targets/RunOneLoopKernelUncutMinus.wl: runs the $-L$ GaugeLink integral.
- targets/CompareOneLoop.wl: computes F_+ , computes J_- , forms the discontinuity, and compares to the analytic result in one fresh run.
- checks/CompareOneLoopFromFiles.wl: reconstructs the delta constraint from previously exported uncut result files and compares coefficient by coefficient. This is convenient, but it can read stale files if the kinematics were changed.
- families/OneLoopKernelAuxDenFamilies.wl: one-loop $+L$ and $-L$ families with one extra auxiliary denominator.
- targets/RunOneLoopKernelAuxDenPlus.wl: runs the auxiliary denominator $+L$ piece with the auxiliary exponent set to zero.
- targets/RunOneLoopKernelAuxDenMinus.wl: runs the auxiliary denominator $-L$ piece with the auxiliary exponent set to zero.
- targets/CompareOneLoopAuxDen.wl: computes both auxiliary denominator pieces and compares to the analytic one-loop kernel.
- checks/CompareOneLoopAuxDenFromFiles.wl: post-processes the exported auxiliary denominator pieces.

- `checks/CompareOneLoopDirectCut.wl`: legacy comparison script for the direct cut-linear attempt.
- `families/TwoLoopKernelUncutFamilies.wl`: four two-loop uncut GaugeLink families, labelled PP, PM, MP, and MM.
- `targets/TwoLoopKernelTargetTools.wl`: shared helper functions for the two-loop targets.
- `targets/RunTwoLoopKernelUncutPP.wl`: runs the PP two-loop sign piece alone.
- `targets/RunTwoLoopKernelUncutPM.wl`: runs the PM two-loop sign piece alone.
- `targets/RunTwoLoopKernelUncutMP.wl`: runs the MP two-loop sign piece alone.
- `targets/RunTwoLoopKernelUncutMM.wl`: runs the MM two-loop sign piece alone.
- `targets/CompareTwoLoop.wl`: computes all four two-loop pieces, forms the double discontinuity, and compares to the closed form.
- `targets/CompareTwoLoopFixedEps.wl`: computes the original AMFlow two-loop double-discontinuity expression once and evaluates the resulting function of ϵ at a fixed value or at an ϵ -list.
- `checks/CompareTwoLoopFromFiles.wl`: post-processes previously exported two-loop sign pieces.
- `../local-subtraction-method/scripts/two_loop_method3_mc_compare.py`: Python driver for the Method-3 transverse numerical integral, the Method-1/2 closed form, and optional AMFlow values.
- `../local-subtraction-method/src/lsmethod/method3.py`: numerical implementation of the Method-3 three-dimensional k, l, θ integral.

The shell entry points are:

```
./run.sh bubble
./run.sh oneloop-kernel-direct-cut
./run.sh oneloop-kernel-uncut-plus
./run.sh oneloop-kernel-uncut-minus
./run.sh compare-oneloop
./run.sh compare-oneloop-from-files
./run.sh oneloop-kernel-aux-plus
./run.sh oneloop-kernel-aux-minus
./run.sh compare-oneloop-aux
./run.sh compare-oneloop-aux-from-files
./run.sh compare-oneloop-direct-cut
./run.sh twoloop-kernel-uncut-pp
./run.sh twoloop-kernel-uncut-pm
./run.sh twoloop-kernel-uncut-mp
./run.sh twoloop-kernel-uncut-mm
./run.sh compare-twoloop
./run.sh compare-twoloop-fixed-eps
./run.sh compare-twoloop-from-files
./run.sh two-loop-method3-mc-compare --all-points --eps 0.1 --amflow skip
```

Target Reference

Target	Role	When to use it
bubble	Ordinary one-loop bubble sanity check.	Use this to verify that AMFlow, Mathematica subprocesses, the reducer, and the dynamic libraries still work. It is not a collinear-kernel test.
oneloop-kernel-direct-cut	Direct cut-linear attempt with $L = n \cdot l + x$ marked as cut.	Diagnostic only. This currently fails in AMFlow's cut-linear/boundary handling and is kept to document the failed direct route.
oneloop-kernel-uncut-plus	Computes F_+ , the uncut GaugeLink integral with denominator $L + i0$.	Diagnostic/cache target. Useful for inspecting the $+L$ piece alone. By itself it is not the delta-constrained kernel.
oneloop-kernel-uncut-minus	Computes J_- , the uncut GaugeLink integral with denominator $-L + i0$.	Diagnostic/cache target. Useful for inspecting the $-L$ piece alone. By itself it is not the delta-constrained kernel.
compare-oneloop	Fresh full one-loop check. Computes F_+ , computes J_- , forms $(-J_- - F_+)/(2\pi i)$, and compares to the analytic kernel.	Preferred target for the actual one-loop physics sanity check. It avoids stale result files because both AMFlow computations and the comparison happen in the same run.
compare-oneloop-from-files	Reads previously exported F_+ and J_- result files, forms the discontinuity, and compares to the analytic expression.	Quick post-processing only. Use with care: it can compare stale data if the result files were generated with old kinematics or old ϵ -order.
oneloop-kernel-aux-plus	Computes F_+ with one extra auxiliary denominator at exponent zero.	Diagnostic target for testing whether adding a zero-power denominator changes the one-loop GaugeLink result.
oneloop-kernel-aux-minus	Computes J_- with one extra auxiliary denominator at exponent zero.	Diagnostic target paired with oneloop-kernel-aux-plus .
compare-oneloop-aux	Fresh auxiliary-denominator check. Computes both pieces, forms $(-J_- - F_+)/(2\pi i)$, and compares to the analytic kernel.	Preferred target for testing the zero-power denominator mechanism without relying on old result files.
compare-oneloop-aux-from-files	Reads previously exported auxiliary F_+ and J_- result files and compares their discontinuity to the analytic expression.	Quick post-processing only. Use only after rerunning both auxiliary pieces with the current setup.
compare-oneloop-direct-cut	Legacy comparison script for the direct cut-linear result.	Not part of the recommended workflow at present, because the direct cut-linear AMFlow run does

4 Stage 1: Bubble Sanity Check

The bubble family is

$$B = \int \frac{d^d l}{i\pi^{d/2}} \frac{1}{(l^2 - m^2)((l + p)^2 - m^2)} \quad (1)$$

at the Euclidean point

$$p^2 = s = -1, \quad m^2 = \frac{1}{10}. \quad (2)$$

This is not a physics target; it is only a sanity check that AMFlow, the reducer, Mathematica subprocesses, and the dynamic libraries work.

The analytic calculation is also simple. Introduce a Feynman parameter,

$$\frac{1}{D_1 D_2} = \int_0^1 dz \frac{1}{[(l + zp)^2 - m^2 + z(1 - z)p^2]^2}. \quad (3)$$

After shifting the loop momentum and using $d = 4 - 2\epsilon$, the normalized one-loop integral becomes

$$B = \Gamma(\epsilon) \int_0^1 dz [m^2 - z(1 - z)p^2]^{-\epsilon}. \quad (4)$$

For the bubble sanity point $p^2 = -1$, $m^2 = 1/10$,

$$B = \Gamma(\epsilon) \int_0^1 dz \left[\frac{1}{10} + z(1 - z) \right]^{-\epsilon}. \quad (5)$$

Expanding,

$$\Gamma(\epsilon) = \frac{1}{\epsilon} - \gamma_E + \mathcal{O}(\epsilon), \quad (6)$$

$$\left[\frac{1}{10} + z(1 - z) \right]^{-\epsilon} = 1 - \epsilon \log \left[\frac{1}{10} + z(1 - z) \right] + \mathcal{O}(\epsilon^2), \quad (7)$$

so

$$B = \frac{1}{\epsilon} - \gamma_E - \int_0^1 dz \log \left[\frac{1}{10} + z(1 - z) \right] + \mathcal{O}(\epsilon). \quad (8)$$

The parameter integral can be evaluated as

$$\int_0^1 dz \log[a + z(1 - z)] = \log a - 2 + 2\sqrt{1 + 4a} \operatorname{artanh} \left(\frac{1}{\sqrt{1 + 4a}} \right), \quad (9)$$

with $a = 1/10$. Therefore

$$B = \frac{1}{\epsilon} + 0.7934919436865079827886470731 \dots + \mathcal{O}(\epsilon). \quad (10)$$

This matches the AMFlow output one-to-one:

$$B_{\text{AMFLOW}} = \frac{1}{\epsilon} + 0.7934919436865079827886470731 \dots + \mathcal{O}(\epsilon). \quad (11)$$

5 Stage 2: Target One-Loop Kernel

The one-loop differential collinear kernel to be tested is

$$I_c(x, m, M) = \int \frac{d^d l}{i\pi^{d/2}} \frac{\delta(x + n \cdot l)}{(l^2 - m^2)((l + p)^2 - M^2)}, \quad d = 4 - 2\epsilon, \quad (12)$$

with

$$n^2 = 0, \quad p \cdot n = 1, \quad p^2 = 0. \quad (13)$$

The symbol `eta` is avoided because AMFlow reserves it for the auxiliary mass-flow variable. The light-like vector is called `n`.

The AMFlow denominator convention is

$$D_1 = l^2 - m^2, \quad D_2 = (l + p)^2 - M^2, \quad L = n \cdot l + x. \quad (14)$$

The active numerical point is

$$x = \frac{1}{3}, \quad m^2 = \frac{1}{10}, \quad M^2 = \frac{1}{5}, \quad p^2 = 0. \quad (15)$$

6 Stage 3: Direct Cut-Linear Attempt

The most literal AMFlow encoding is to include $L = n \cdot l + x$ as the third propagator and set

$$\text{AMFlowInfo}["\text{Cut}"] = \{0, 0, 1\}. \quad (16)$$

This is implemented in `OneLoopKernelDirectCutFamily.wl`. It was useful as a diagnostic, but it is not currently the working route:

- ordinary `SolveIntegrals` fails immediately because the standard path tries to rewrite all denominators as squared denominators;
- `SolveIntegralsGaugeLink` accepts the linear denominator, but with the cut flag it fails later at an ending-system/boundary stage.

Thus the syntax $n \cdot l + x$ is not the fundamental issue. The obstacle is direct automatic treatment of a cut linear denominator.

7 Stage 4: GaugeLink Linear-Propagator Tests

AMFlow has a special function `SolveIntegralsGaugeLink` for ordinary linear propagators. It implements the strategy described in the AMFlow linear propagator paper: introduce an auxiliary quadratic representation for a linear denominator, solve differential equations in the auxiliary parameter, and take the relevant limit.

We first tested the uncut integral

$$F_+(x) = \int \frac{d^d l}{i\pi^{d/2}} \frac{1}{D_1 D_2 (L + i0)}. \quad (17)$$

This has no direct delta-constraint meaning by itself, but it verifies that AMFlow can handle the linear denominator machinery. The test succeeded.

The $+L$ and $-L$ families are kept as separate AMFlow family files because AMFlow's reduction tables, cache directories, and integral heads are tied to a single family definition. This separation was also useful during debugging. However, a physics comparison should not rely on manually combining result files from different runs, because those files may have been generated with older kinematics. The preferred target is therefore

`./run.sh compare-oneloop`

which performs both GaugeLink calculations and the comparison in one fresh Mathematica session.

8 Stage 5: Discontinuity Reconstruction

Rather than cutting L directly in AMFlow, we reconstruct the delta function from two ordinary linear-propagator integrals. The identity used is

$$\delta(L) = \frac{1}{2\pi i} \left(\frac{1}{L - i0} - \frac{1}{L + i0} \right). \quad (18)$$

The second AMFlow run uses the denominator $-L$:

$$J_-(x) = \int \frac{d^d l}{i\pi^{d/2}} \frac{1}{D_1 D_2(-L + i0)}. \quad (19)$$

Since

$$\frac{1}{L - i0} = -\frac{1}{-L + i0}, \quad (20)$$

the physical discontinuity candidate is

$$I_c^{\text{disc}}(x, m, M) = \frac{-J_-(x) - F_+(x)}{2\pi i}. \quad (21)$$

The comparison script also prints sign variants, but the expression above is the intended one.

9 Analytic Formula

For the routing used here,

$$I_c(x, m, M) = \frac{\Gamma(1 + \epsilon)}{\epsilon} A^{-\epsilon} \Theta(0 \leq x \leq 1), \quad (22)$$

where

$$A = (1 - x)m^2 + xM^2 - x(1 - x)p^2. \quad (23)$$

With the intended light-like value $p^2 = 0$,

$$A = (1 - x)m^2 + xM^2. \quad (24)$$

At the active point,

$$A = \frac{2}{3} \frac{1}{10} + \frac{1}{3} \frac{1}{5} = \frac{2}{15}. \quad (25)$$

Therefore the expected expansion is

$$\frac{\Gamma(1 + \epsilon)}{\epsilon} \left(\frac{2}{15} \right)^{-\epsilon}. \quad (26)$$

The finite coefficient is

$$\log\left(\frac{15}{2}\right) - \gamma_E. \quad (27)$$

10 Important Correction: p^2

An earlier one-loop run accidentally used $s = p^2 = -1$. This came from the Euclidean bubble test and was not the intended collinear kinematics. With $s = -1$, the discontinuity check correctly matched the finite coefficient for

$$A = (1-x)m^2 + xM^2 - x(1-x)s = \frac{16}{45}, \quad (28)$$

which was a useful sign/normalization check, but not the desired physics point.

The three one-loop family files now use

$$s \rightarrow 0. \quad (29)$$

Consequently, one-loop result files produced before this correction are stale.

11 Current High-Order Test

The preferred current high-order test is the single fresh-run target

```
./run.sh compare-oneloop
```

It computes F_+ , computes J_- , forms $(-J_- - F_+)/(2\pi i)$, and compares directly to the analytic expression.

The older split workflow is still available for diagnostics:

```
./run.sh oneloop-kernel-uncut-plus
./run.sh oneloop-kernel-uncut-minus
./run.sh compare-oneloop-from-files
```

but the last command reads exported files and can therefore compare stale data if one of the two calculations was not rerun after a configuration change.

The two GaugeLink target scripts have been set to

$$\text{epsOrder} = 5. \quad (30)$$

For `SolveIntegralsGaugeLink`, this is necessary because AMFlow internally uses a one-loop leading power -2 , so the returned expansion reaches $\epsilon^{\text{epsOrder}-2}$. Thus `epsOrder = 5` returns coefficients through ϵ^3 . The comparison script expands the analytic result through ϵ^3 and compares powers

$$\epsilon^{-1}, \epsilon^0, \epsilon^1, \epsilon^2, \epsilon^3. \quad (31)$$

The comparison is coefficient-wise:

$$c_p^{\text{AMFlow}} = [\epsilon^p] \frac{-J_- - F_+}{2\pi i}, \quad c_p^{\text{ana}} = [\epsilon^p] \frac{\Gamma(1+\epsilon)}{\epsilon} A^{-\epsilon}. \quad (32)$$

For each p , the script prints

$$c_p^{\text{AMFlow}}, \quad c_p^{\text{ana}}, \quad c_p^{\text{AMFlow}} - c_p^{\text{ana}}, \quad \frac{c_p^{\text{AMFlow}}}{c_p^{\text{ana}}}. \quad (33)$$

12 Stage 5b: One-Loop Auxiliary-Denominator Test

The two-loop family needs an extra denominator to complete the scalar-product space. It is then requested with exponent zero. Before trusting this mechanism in the expensive two-loop run, the same idea is tested in the one-loop setup. The auxiliary one-loop families use

$$D_1 = l^2 - m^2, \quad D_2 = (l + p)^2 - M^2, \quad D_3 = \pm(n \cdot l + x), \quad D_4 = (l - p)^2. \quad (34)$$

The AMFlow targets are

$$\text{j}[\text{oneloopkernelauxplus}, 1, 1, 1, 0], \quad \text{j}[\text{oneloopkernelauxminus}, 1, 1, 1, 0]. \quad (35)$$

Thus D_4 is present in the family definition but absent from the integral. The physical one-loop integral should therefore be unchanged.

The fresh target is

`./run.sh compare-oneloop-aux`

It computes the auxiliary F_+ and J_- pieces, forms the same discontinuity,

$$I_{c,\text{aux}}^{\text{disc}} = \frac{-J_-^{\text{aux}} - F_+^{\text{aux}}}{2\pi i}, \quad (36)$$

and compares it coefficient by coefficient to

$$\frac{\Gamma(1 + \epsilon)}{\epsilon} A^{-\epsilon}, \quad A = (1 - x)m^2 + xM^2 - x(1 - x)p^2. \quad (37)$$

The split workflow is also available:

`./run.sh oneloop-kernel-aux-plus`

`./run.sh oneloop-kernel-aux-minus`

`./run.sh compare-oneloop-aux-from-files`

The file-based command is only post-processing. It must not be trusted after a change of kinematics or ϵ -order unless both auxiliary pieces were rerun.

13 Stage 6: Two-Loop GaugeLink Setup

The two-loop target is the double-collinear kernel from `derivations/snippets/dckernel/main.tex`:

$$\mathcal{I}(x, y) = \int \frac{d^d k}{i\pi^{d/2}} \frac{d^d l}{i\pi^{d/2}} \frac{\delta(x - l \cdot n) \delta(y - k \cdot n)}{(l^2 - m_l^2)((l - p)^2 - M_l^2)(k^2 - m_k^2)((l + k - p)^2 - M_k^2)}. \quad (38)$$

The AMFlow setup uses

$$n^2 = 0, \quad p \cdot n = 1, \quad (39)$$

and the two linear denominators

$$L_x = x - n \cdot l, \quad L_y = y - n \cdot k. \quad (40)$$

As in the one-loop test, the direct cut-linear route is not used. Instead, four uncut GaugeLink pieces are introduced:

$$I_{PP} : \frac{1}{(L_x + i0)(L_y + i0)}, \quad (41)$$

$$I_{PM} : \frac{1}{(L_x + i0)(-L_y + i0)}, \quad (42)$$

$$I_{MP} : \frac{1}{(-L_x + i0)(L_y + i0)}, \quad (43)$$

$$I_{MM} : \frac{1}{(-L_x + i0)(-L_y + i0)}. \quad (44)$$

The double delta constraint is reconstructed from

$$\delta(L_x)\delta(L_y) = \frac{1}{(2\pi i)^2} \left(\frac{1}{L_x - i0} - \frac{1}{L_x + i0} \right) \left(\frac{1}{L_y - i0} - \frac{1}{L_y + i0} \right). \quad (45)$$

Using

$$\frac{1}{L - i0} = -\frac{1}{-L + i0}, \quad (46)$$

the implemented double-discontinuity candidate is

$$\mathcal{I}_{\text{disc}} = \frac{I_{PP} + I_{PM} + I_{MP} + I_{MM}}{(2\pi i)^2}. \quad (47)$$

The first numerical point is now a light-like point chosen to avoid branch phases while debugging the discontinuity normalization:

$$x = \frac{3}{10}, \quad y = \frac{1}{4}, \quad p^2 = 0, \quad (48)$$

and

$$m_l^2 = \frac{49}{100}, \quad M_l^2 = 4, \quad m_k^2 = \frac{1}{4}, \quad M_k^2 = \frac{81}{100}. \quad (49)$$

This differs from the older Euclidean Python-check point. Both Δ_0 and Δ_1 are positive at this point, so the first AMFlow comparison is not mixed with a branch-cut phase test. The analytic comparison uses

$$\mathcal{I}_{\text{closed}} = \frac{\Gamma(\epsilon)^2}{1-x} [\Delta_0 - i0]^{-\epsilon} [\Delta_1 - i0]^{-\epsilon}, \quad (50)$$

with

$$\Delta_0 = (1-x)m_l^2 + xM_l^2 - x(1-x)p^2, \quad (51)$$

and

$$\Delta_1 = -[(1-\lambda)m_k^2 + \lambda M_k^2] + \lambda(1-\lambda)M_l^2 + \lambda(1-x)p^2, \quad \lambda = \frac{y}{1-x}. \quad (52)$$

At the current stage the two-loop targets use

$$\text{precisionGoal} = 10, \quad \text{epsOrder} = 4, \quad (53)$$

and compare the powers

$$\epsilon^{-2}, \epsilon^{-1}, \epsilon^0. \quad (54)$$

This is still a modest run, but it should include the finite coefficient.

13.1 Method-3 Transverse Numerical Check

The target

`./run.sh two-loop-method3-mc-compare`

is a Python-driven comparison target. It lives in `../local-subtraction-method/scripts/two_loop_method3` and can compare three objects:

1. the Method-3 transverse numerical representation;
2. the Method-1/2 closed form $\Gamma(\epsilon)^2(1-x)^{-1}[\Delta_0 - i0]^{-\epsilon}[\Delta_1 - i0]^{-\epsilon}$;
3. the original AMFlow two-loop double-discontinuity expression, if `--amflow fresh` is requested.

The fixed- ϵ mode is, for example,

```
./run.sh two-loop-method3-mc-compare --all-points --eps 0.1 --amflow skip
```

and the coefficient-fit mode is

```
./run.sh two-loop-method3-mc-compare --all-points \
  --compare-coefficients --amflow skip
```

On the cluster the same target can be called through the Slurm wrapper, e.g.

```
AMFLOW_TARGET=two-loop-method3-mc-compare \
AMFLOW_ARGS="--point equal_mass_onshell_branch --eps 0.1 --amflow fresh" \
sbatch run_amflow.slurm
```

The Method-3 implementation deliberately does not perform the angular integration analytically. The remaining integral is the genuine three-dimensional integral

$$\int_0^\infty dk \int_0^\infty dl \int_0^\pi d\theta \frac{k^{1-2\epsilon} l^{1-2\epsilon} (\sin \theta)^{-2\epsilon}}{D_2(l) D_4(l, k, \theta)}. \quad (55)$$

For the equal-mass and $p_\perp = 0$ check points the denominators are implemented as

$$D_2(l) = (y - p^+) \left(\frac{l^2 + M_l^2}{y} - p^- \right) - (l^2 + M_l^2) + i\eta \frac{p^+}{y}, \quad (56)$$

$$D_4(l, k, \theta) = A(l, k; x, y) - 2lk \cos \theta + i\eta \Gamma_{xy}, \quad (57)$$

where

$$\Gamma_{xy} = \frac{(x + y)(p^+ - x - y)}{xy} \quad (58)$$

and

$$A(l, k; x, y) = \frac{x - p^+}{y} l^2 + \frac{y - p^+}{x} k^2 + M^2 \frac{(x + y)(x + y - p^+) - xy}{xy} - (x + y - p^+) p^-. \quad (59)$$

The equal-mass assumption is explicit in the code. The old AMFlow point **branch_safe_rational** is non-equal-mass, so the Python target can still run Method 1/2 and AMFlow for that point, but it skips Method 3 rather than silently applying the equal-mass formula outside its domain.

The infinite radial intervals are mapped to the unit cube by

$$k = \rho \frac{u}{1 - u}, \quad l = \rho \frac{v}{1 - v}, \quad \theta = \pi w, \quad 0 < u, v, w < 1. \quad (60)$$

The Jacobian used in **method3.py** is

$$dk dl d\theta = \frac{\rho}{(1 - u)^2} \frac{\rho}{(1 - v)^2} \pi du dv dw. \quad (61)$$

The default scale is $\rho = \sqrt{M^2}$. The command-line option **--rho** can be used to test the dependence on this map scale.

The unit-cube integral is evaluated by quasi-Monte-Carlo. If SciPy is available, the code uses scrambled Sobol points in three dimensions and therefore expects N to be a power of two. If SciPy is unavailable, it falls back to a shifted Halton sequence. Multiple scrambles are controlled by **--scrambles**; the printed error estimate is the standard error of the mean over the scramble means. With **--scrambles 1** no statistical error can be estimated, so the error column is **nan**. The sample count is controlled by **--N**, for example

```
./run.sh two-loop-method3-mc-compare --point equal_mass_onshell_branch \
--eps 0.1 --N 262144 --scrambles 4 --amflow skip
```

The Method-3 normalization is applied only after the stripped numerical integral is computed. The prefactor printed in debug mode is

$$(2\pi i)^2 \frac{\Omega_{n-1}\Omega_{n-2}}{(i\pi^{d/2})^2} \frac{1}{xy}, \quad d = 4 - 2\epsilon, \quad n = d - 2, \quad (62)$$

with

$$\Omega_m = \frac{2\pi^{m/2}}{\Gamma(m/2)}. \quad (63)$$

The factor $\Omega_{n-2} = \Omega_{-2\epsilon}$ is kept as the analytically continued solid-angle factor; it is not replaced by a two-dimensional angular measure.

The option `--debug` prints the formula components, the Method-3 normalization, fixed deterministic integrand probes, finite radial-cutoff diagnostics, ρ -scans, N -scans, and η -scans. The purpose is to diagnose formula, branch, normalization, mapping, and convergence issues without introducing UV subtraction and without replacing the θ integral by the analytic angular function.

13.2 Fixed- ϵ AMFlow Helper Target

The file `targets/CompareTwoLoopFixedEps.wl` is the Mathematica helper used by the Python comparison target. It loads the same two-loop family definitions as `CompareTwoLoop.wl`, solves the four AMFlow sign pieces for the selected kinematic point, forms

$$\mathcal{I}_{\text{disc}}(\epsilon) = \frac{I_{PP}(\epsilon) + I_{PM}(\epsilon) + I_{MP}(\epsilon) + I_{MM}(\epsilon)}{(2\pi i)^2}, \quad (64)$$

and then evaluates this expression at requested positive ϵ -values.

For a single fixed- ϵ check, the Python driver sets `TWO_LOOP_EPS_VALUE`. For coefficient fits it sets `TWO_LOOP_EPS_LIST`, for example

```
TWO_LOOP_EPS_LIST="1/5,3/20,1/10,3/40,1/20"
```

In that case AMFlow is run only once for the selected point; the resulting expression in ϵ is reused for every value in the list. This avoids rerunning the expensive AMFlow reduction/evaluation separately for each ϵ . The Python driver parses the lines

```
AMFLOW_EPS_LIST_VALUE_INDEX=...
```

from the helper target and uses those values in the same Laurent-fitting machinery as Method 3.

All output from the Python target is written to

```
logs/two-loop-method3-mc-compare.log
```

and the internal AMFlow helper runs are additionally saved as

```
logs/two-loop-method3-mc-compare_amflow_<point>_eps_<eps>.log
```

```
logs/two-loop-method3-mc-compare_amflow_<point>_eps_list_<epslist>.log
```

Coefficient and debug reports are written under `targets/output/`.

For quick smoke tests, the same scripts can be run at lower precision while keeping the same ϵ -order. The predefined shortcut

```
./run.sh compare-twoloop-quick
```

sets

$$\text{precisionGoal} = 6, \quad \text{epsOrder} = 4, \quad (65)$$

unless these values are overridden by environment variables. This should be used only to catch setup mistakes and coefficient-level problems at rough precision. A slightly more accurate check is, for example,

```
AMFLOW_PRECISION_GOAL=8 ./run.sh compare-twoloop-quick
```

The single-piece shortcut

```
./run.sh twoloop-kernel-uncut-pp-quick
```

is useful when testing whether one representative family still reduces.

The first completed run with the older $p^2 = -3$ point produced a leading coefficient exactly one half of the closed-form value. The comparison scripts therefore now print an inferred leading-pole normalization,

$$\frac{[\epsilon^{-2}]\mathcal{I}_{\text{disc}}}{[\epsilon^{-2}]\mathcal{I}_{\text{closed}}}, \quad (66)$$

and expose a variable `doubleCutNormalization`. It is initially set to 1. If the light-like rerun again gives a constant factor $1/2$, set `doubleCutNormalization = 1/2` in the comparison script and rerun only the post-processing comparison. No AMFlow recomputation is needed for that normalization check.

The recommended debugging order is:

```
./run.sh twoloop-kernel-uncut-pp
```

If that succeeds, try the remaining pieces or the fresh full check:

```
./run.sh compare-twoloop
```

For quick post-processing after all four sign pieces have already been exported, use:

```
./run.sh compare-twoloop-from-files
```

The file-based comparison can read stale data, so it should not be used after changing the kinematic point or ϵ -order unless all four pieces were rerun.